8. Level 3: Component diagram

Following on from a Container diagram showing the high-level technology elements, the next step is to zoom in and decompose each container further to show the components inside it.

8.1 Intent

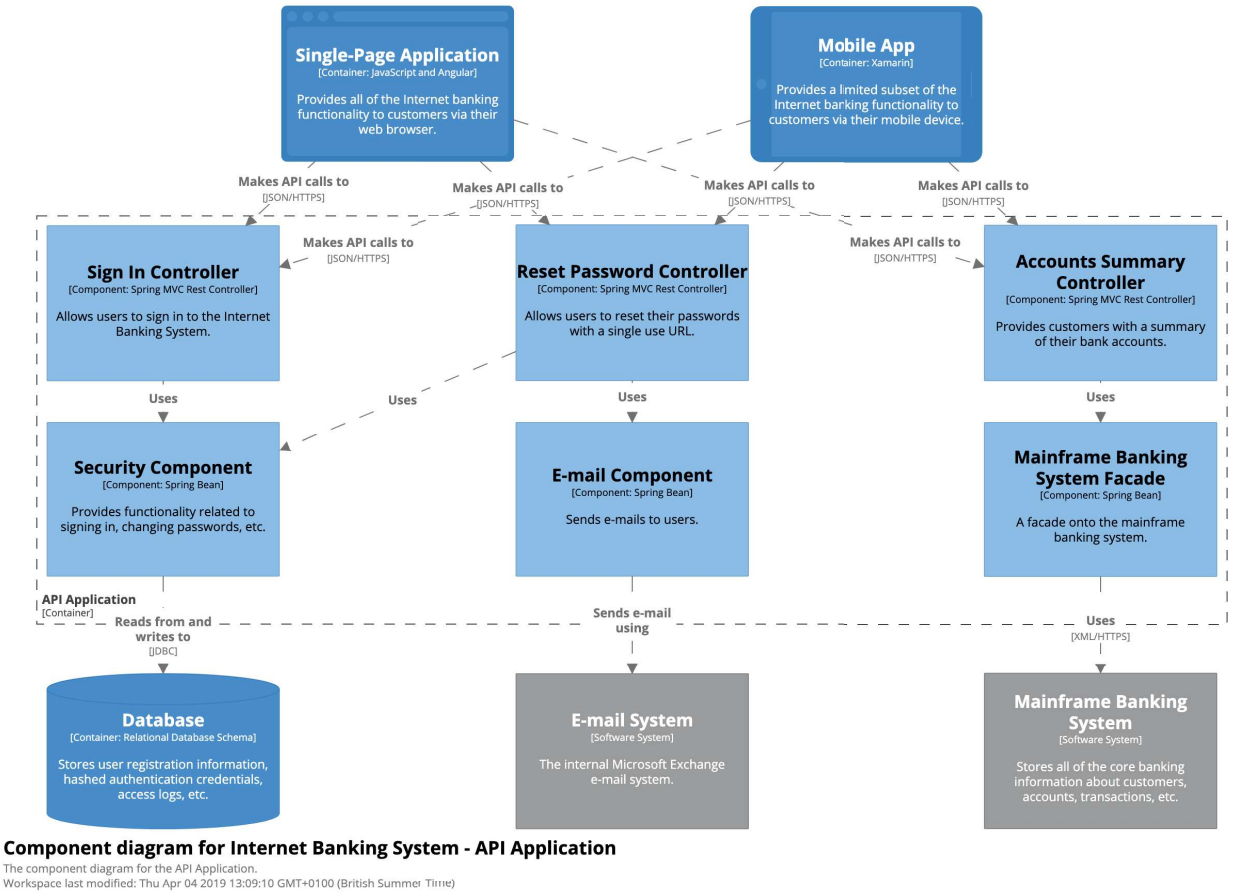
A Component diagram helps you answer the following questions.

1. What components is each container made up of?
2. Do all components have a home (i.e. reside in a container)?
3. It is clear how the software works at a high-level?

8.2 Structure

Whenever people are asked to draw “architecture diagrams”, they usually end up drawing diagrams showing the components that make up their software system. This is basically what a Component diagram shows, except we only want to see the components that reside within a *single* container at a time.

This is an example Component diagram for the fictional Internet Banking System, showing some (rather than all) of the components within the API Application.



Component diagram for Internet Banking System - API Application

The component diagram for the API Application.
Workspace last modified: Thu Apr 04 2019 13:09:10 GMT+0100 (British Summer Time)

Here, there are three Spring MVC Rest Controllers providing access points for the JSON/HTTPS API, with each controller subsequently using other components to access data from the Database and Mainframe Banking System, or send e-mails. In summary, this diagram shows how the API Application is divided into components, what each of those components are, their responsibilities and the technology/implementation details.

How much detail?

If you're drawing a Component diagram during an up-front design exercise, you might not have some of the technical details to hand. Once again, don't worry, add what you know. If, on the other hand, you're drawing a diagram to document an existing system, you'll have those finer details to hand; such as the frameworks you are using to help implement a component. As with many other aspects of the diagrams, the choice of how much detail to include is yours.

8.3 Elements

A Component diagram can include four types of elements; people, software systems, containers and components.

Components

As we saw when creating our shared vocabulary, components are the coarse-grained building blocks of your software system that live inside of a container. I'll capture the following information for each component:

- **Name**: The name of the component.
- **Technology**: The implementation technology for the component (e.g. Plain Old [Java|C#|Ruby|etc] Object, Enterprise JavaBean, Windows Communication Foundation service, etc).
- **Description**: A short description of the component, usually a brief sentence describing the component's responsibilities.

You can think about and identify components regardless of how the code is packaged and the architectural style in use. With this in mind, your Component diagram should reflect the architectural style in use; whether that's a layered architecture, hexagonal architecture or something else entirely.

Infrastructure components and cross-cutting concerns

Infrastructure components are important parts of most software systems, yet you may or may not want to include them on your Component diagram. For example, if you have a logging component, it's likely to be used by the majority of other components within the container. Drawing this component and all of the interactions can result in a very cluttered diagram. You have a number of options to deal with this:

1. Don't include the logging component if it doesn't add much value in helping you tell the story.
2. Write a note on the diagram that says something like, "All components shown here write logs via a Logging Component".
3. Include the logging component on the diagram, but don't show how it's connected to anything else. Instead, annotate elements that use the logging component (with a symbol/icon), or use a colour coding, which you can then describe in a diagram key/legend (e.g. "the asterisk denotes a relationship with the Logging Component").

Shared components and libraries

I'm often asked whether a Component diagram should include shared components (for example, from a shared or static library) and how such components should be represented. If including the shared component helps tell the story, then it should certainly be included. If you want to illustrate that a particular component is a shared component or sits within a specific library/module, again, you can use a notation (e.g. a symbol/icon or colour coding) to represent this fact.

Multiple component implementations

In many cases, there will only ever be a single implementation of a component. However, there may be times when you'll have a single component interface and a number of implementations. This is particularly true of software *products* (rather than bespoke software), where the collection of active components will be selected through configuration when the product is installed. Common examples include different implementations of data storage components (e.g. one for Microsoft SQL Server, one for MySQL, etc), different logging components (e.g. local disk or a message queue), or pluggable authentication components for integration with different identity providers.

Having multiple component implementations raises the question of how this should be illustrated on a Component diagram. If we take an example of a logging component with multiple implementations (e.g. local disk vs a message queue), one of which is chosen at deployment time via configuration, there are a few approaches to diagramming this:

1. The first approach is to omit the fact that there are multiple component implementations and draw a Component diagram as if there was only a single implementation. Here, I would draw the logging component and describe it's responsibilities, which in this case might be to "log errors and other system events".
2. If I wanted to include the fact that the component implementation can vary, I might add some additional text to the logging component box on the diagram to say something like "Log entries are stored using local disk or sent to a message queue, depending upon the implementation chosen by configuration at deployment time". You could also achieve the same result by highlighting the logging component using a symbol/icon or colour coding. The diagram key would then explain this.
3. The other approach is to have one Component diagram per implementation option. This works best if you only have a small number of component implementation combinations (e.g. you only have one or two components where the implementation can be swapped in at runtime). Having separate diagrams for specific component

implementations can also be useful if those component implementations themselves introduce other components, which wouldn't be seen on a diagram otherwise.

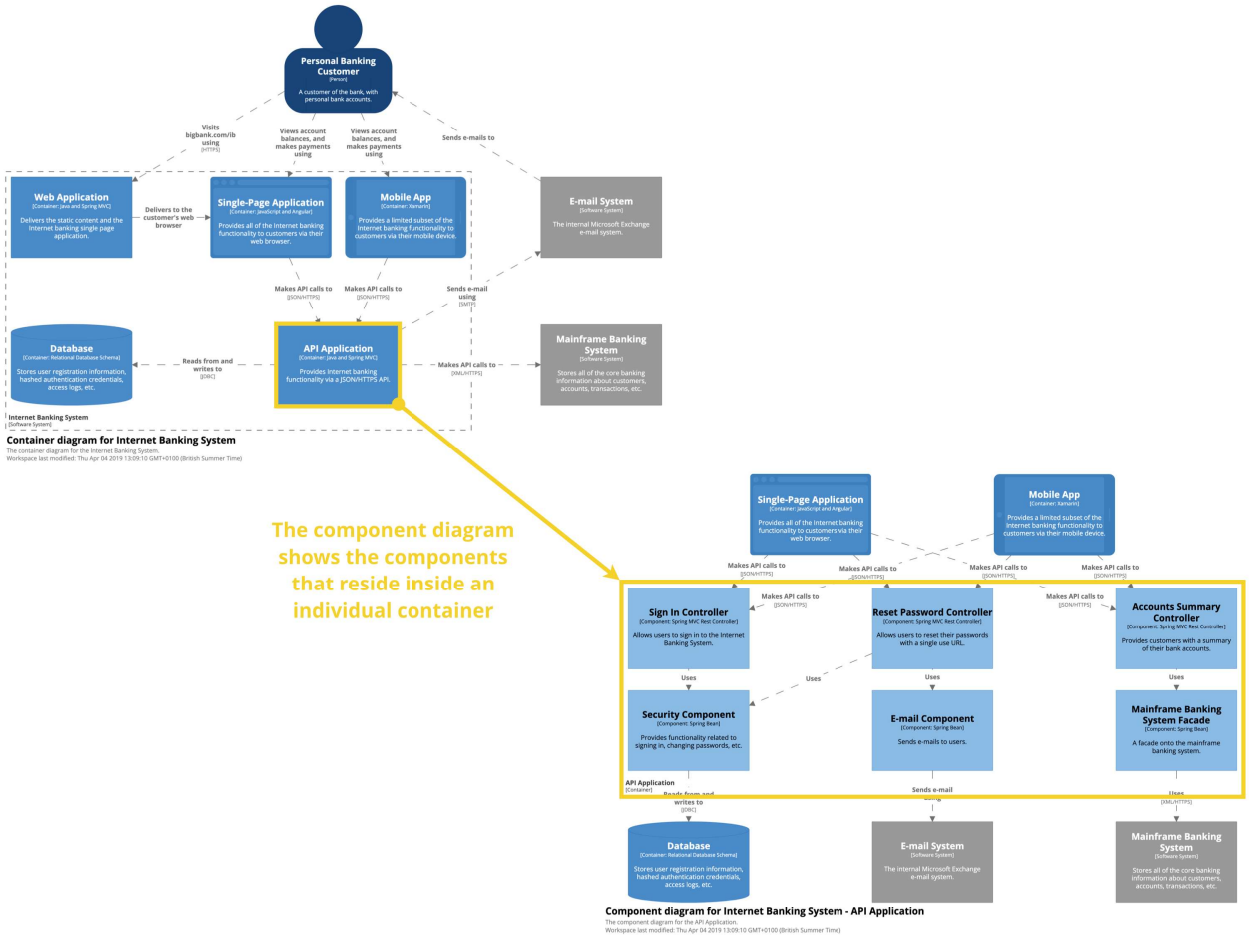
As with many of the things discussed in this book, there is no "right" answer, and it really depends on what story you need to tell.

People, software systems and containers

As with the Container diagram, it can be useful to include people, software systems and other containers to help put some context around the container you've zoomed in upon. The Internet Banking System Component diagram for the API Application includes the other containers and software systems that it interacts with.

Container boundary

If you're going to include people, software systems and containers on a Component diagram (and, again, you should), I recommend drawing a bounding box to explicitly show the boundary of the container.



The Component diagram shows a zoom in of a single container

8.4 Interactions

To reiterate the same advice given for other diagram types, it's useful to annotate the interactions between components rather than having a diagram with a collection of boxes and ambiguous lines connecting them all together. Useful information to add the diagram includes:

- The purpose of the interaction (e.g. "uses", "persists data using", "delegates to", etc).
- Communication style (e.g. synchronous, asynchronous, etc).

8.5 Motivation

A Component diagram shows the components that reside inside an individual container. This is useful because:

- It shows the high-level decomposition of a container into components, each with distinct responsibilities.
- It shows where there are relationships and dependencies between components.
- It provides a high-level summary of the implementation details, including any frameworks or libraries being used.
- If the components shown on the diagram can be explicitly mapped to the code, you have a good way to really understand the structure of a codebase.

8.6 Audience

Technical people within the software development team.

8.7 Required or optional?

I don't typically draw a Component diagram for data storage containers (e.g. databases, file systems, content stores, etc), or for those containers that are very simple in nature (e.g. microservices). For monolithic applications, you might consider creating a Component diagram, especially during an up front design process. In real world use, many teams find this an optional level of detail for long-lived documentation, because of the potentially high degree of effort involved in creating the diagrams, and keeping them up to date. Also, once you have more than a handful of components, you'll find that the diagram starts to get very cluttered very quickly, which reduces the usefulness of the diagram. We'll look at some strategies for dealing with this later in the book.